MACHINE LEARNING

머신러닝 (Machine Learning)

지도학습 알아보기 — KNN & 선형 회귀

2026 UPDATE | 머신러닝 기초 과정 · ch02 지도학습(회귀)

CONTENTS

학습 목차

01 머신러닝 (Machine Learning) 개요

02 지도학습 — 분류와 회귀

03 과대적합 · 과소적합 · 일반화

04 k-최근접 이웃 (k-NN)

05 선형 모델 & 선형 회귀

06 하이퍼파라미터

07 릿지 (Ridge) / 라쏘 (Lasso) 회귀

08 ElasticNet (엘라스틱넷)

머신러닝 (Machine Learning) 이란?



지도학습 (Supervised)

예측하려는 값(타깃)이 있는 학습
- 분류 (Classification)
- 회귀 (Regression)



비지도학습 (Unsupervised)

타깃(정답)이 없는 학습
+ 준지도학습, 강화학습 등



머신러닝의 최종 목표 = 훈련 세트에서 학습하고 새로운 데이터에서도 잘 예측하는 것 (일반화)

지도학습 — 분류와 회귀



분류 (Classification)

미리 정의된 클래스 레이블 중 하나를 예측
이진 분류: 예/아니오, 생존/사망
다중 분류: 3개 이상의 클래스



회귀 (Regression)

타깃이 연속적인 숫자일 때 예측
예: 주택 가격 예측, 주가 예측
오차 지표: MSE, MAE, RMSE



회귀 = 숫자 예측 / 분류 = 카테고리 예측 | 둘 다 '정답(타깃)이 있는' 지도학습입니다.

과대적합 · 과소적합 · 일반화

과대적합 (Overfitting)

훈련 데이터에 너무 맞춰져
새 데이터에서 성능이 떨어짐

일반화 (Generalization)

훈련·테스트 모두 안정적으로
좋은 성능을 내는 상태 — 목표!

과소적합 (Underfitting)

너무 단순한 모델 선택으로
훈련 데이터조차 잘 학습 못함

해결책: 규제(Regularization), 데이터 증강, 앙상블, 교차 검증(Cross-Validation), Early Stopping

CHAPTER 04

k-최근접 이웃 (k-Nearest Neighbors)

핵심 아이디어

"새 데이터가 주어졌을 때, 가장 가까운 K개의 이웃을 찾아 다수결(분류) 또는 평균(회귀)으로 예측"

01 훈련

모델 파라미터를 학습하지 않고
훈련 데이터를 그대로 저장

02 거리 계산

예측 단계에서 새 데이터와
모든 훈련 데이터의 거리 계산

03 예측

k개의 이웃 레이블로
다수결(분류) 또는 평균(회귀)

KNN = 'Lazy Learning' — 별도의 학습 없이 저장만 하고, 예측 시에 거리를 계산합니다.

KNN — 분류 vs 회귀

KNN 분류

k개 이웃의 레이블 중 다수결로 클래스 결정
k=1: 가장 가까운 1개의 이웃 클래스
k=3: 3개 이웃 중 가장 많은 클래스로 결정

KNN 회귀

k개 이웃의 타깃 값의 평균으로 예측
특성 스케일이 다르면 거리 왜곡 발생
→ 정규화(Normalization) 전처리 필수!

KNN은 특성 간 스케일이 다를 경우 거리 계산이 왜곡됩니다 — 반드시 스케일링 먼저!

KNN 모델의 장점 & 단점



장점

구현이 매우 간단하고 직관적
많이 조정하지 않아도 좋은 성능 발휘
분류 & 회귀 모두 적용 가능



단점

수백 개 이상의 특성에선 성능 저하
특성 값의 대부분이 0인 경우 비효율적
스케일이 다를 경우 왜곡 발생 (스케일링 필요)
고차원에서는 거리 개념이 희석 (차원의 저주)



KNN은 입문자에게 적합한 모델! 단, 고차원·대용량 데이터에서는 다른 알고리즘을 검토하세요.

실전에서의 KNN — Kaggle · 산업 사례



Kaggle · 대회

대회 초반 기준선(baseline)으로 빠르게 사용
정형 데이터 상위권은 트리 앙상블이 지배 (LightGBM 등)
본 과정 ch07 — AMEX · Playground 자료와 연결



산업 실무 · 해석

금융·의료: 해석 가능한 릿지/라쏘 모델 선호
KNN은 추천·유사 검색 등 최근접 영역에서 유효
예: 주택 가격(California Housing), 수요 예측 (bike)

KNN·선형 회귀는 '빠른 기준선 + 해석 가능'이 강점 | 실제 대회에서는 조합(앙상블)으로 성능을 올립니다.

선형 모델 (Linear Model)

$$y(\text{예측값}) = w1*x1 + w2*x2 + \ldots + wp*xp + b$$

- **y** 모델이 만들어낸 예측값
- **x** 특성(feature)
- **w** 각 특성에 대한 가중치(기울기)
- **b** 편향(offset / 절편)

특성 1개 → 직선 / 2개 → 평면 / 더 높은 차원 → 초평면으로 결정 경계를 만듭니다.

하이퍼파라미터 (Hyperparameter)

모델 파라미터 (계수)

알고리즘이 데이터로부터 스스로 학습하는 파라미터
예) 선형 회귀의 기울기 w, 절편 b

하이퍼파라미터 (매개변수)

모델이 학습할 수 없어 사람이 직접 설정해야 하는 값
예) KNN의 k 값, 릿지 회귀의 alpha 값

파라미터 = 모델이 스스로 찾음 | 하이퍼파라미터 = 우리가 직접 설정하는 값

선형 회귀 (Linear Regression)

선형 회귀는 평균제곱오차 (MSE) 를 최소화하는 파라미터 (w, b) 를 찾습니다.

$$MSE = (1/n) \times \Sigma (y_실제값 - y_예측값)^2$$

장점

조정할 하이퍼파라미터가 없어
간단하고 빠르게 학습합니다.

단점

복잡도를 제어할 방법이 없어
과대적합 위험이 있습니다.

⚠ 과대적합 문제를 해결하기 위해 정규화 항(Regularization Term)을 추가한 릿지/라쏘를 씁니다.

모델 성능 지표 — 결정계수 R^2

결정계수 (R^2) = 회귀 모델의 예측 적합도

$$R^2 = 1 - \left[\Sigma(y - \hat{y})^2 / \Sigma(y - \bar{y})^2 \right]$$

y = 실제 타깃값 $\hat{y}$ = 모델의 예측값 $\bar{y}$ = 타깃값의 평균

R^2 해석: 0에 가까울수록 예측 불가 → 1에 가까울수록 완벽한 예측 (scikit-learn: model.score())

R^2=1이면 완벽한 예측, R^2=0이면 모델이 평균만 예측하는 것과 같습니다.

정규화항을 통한 일반화

과대적합 해소 → 오차함수에 정규화 항(Regularization term)을 추가

비용함수 = MSE + α × 정규화항(Regularization)

L1 규제 (Lasso)

가중치의 절댓값 합을 패널티로 추가
w의 일부를 정확히 0으로 만듦 → 특성 선택 효과

L2 규제 (Ridge)

가중치의 제곱합을 패널티로 추가
w의 모든 원소를 0에 가깝게 만듦 (0은 안 됨)

정규화 강도 alpha: 클수록 규제 강함 → 모델 단순화 / 작을수록 규제 약함 → 선형 회귀에 가까움

릿지 회귀 (Ridge) — L2 규제

$$\text{비용함수} = \Sigma(y - \hat{y})^2 + \lambda \times \Sigma(wj)^2$$

1. **가중치 크기 제한** — 가중치(w)의 절댓값을 가능한 작게 만들어 과대적합을 방지합니다.
2. **특성 선택 X** — 모든 특성을 0에 가깝게 만들지만, 정확히 0이 되지는 않습니다.
3. **alpha 파라미터** — 데이터가 충분하면 alpha 영향이 줄어듭니다. 기본값: alpha=1

릿지: 모든 특성을 유지하면서 가중치 크기를 줄이고 싶을 때 사용하세요.

라쏘 회귀 (Lasso) — L1 규제

$$\text{비용함수} = \Sigma(y - \hat{y})^2 + \lambda \times \Sigma|w_j|$$

1. **특성 선택 자동화** — 일부 가중치(w)를 정확히 0으로 만들어 불필요한 특성을 제거합니다.
2. **모델 단순화** — 가장 중요한 특성만 남아 모델이 이해하기 쉬워집니다.
3. **alpha 파라미터** — alpha가 너무 크면 모든 계수가 0이 될 수 있습니다.

라쏘: 특성이 많을 때 중요한 특성만 자동으로 골라내고 싶을 때 사용하세요 (Feature Selection).

릿지 vs 라쏘 — 한눈에 비교

구분	릿지 (Ridge) — L2	라쏘 (Lasso) — L1
규제 방식	제곱합 (L2)	절댓값 합 (L1)
가중치 w	0에 가깝게 (0은 X)	일부를 정확히 0으로
특성 선택	하지 않음	자동으로 수행
사용 상황	대부분의 경우 기본 선택	불필요한 특성 많을 때

모를 땐 릿지부터! 특성이 너무 많거나 해석이 필요하면 라쏘를 고려하세요.

ElasticNet (엘라스틱넷)

Ridge + Lasso를 결합한 모델

L1과 L2 두 가지 규제를 동시에 적용하여 장점을 모두 취합니다.
단점: 조정해야 할 하이퍼파라미터가 2개 (alpha, l1_ratio)

선형 회귀	릿지 (L2)	라쏘 (L1)	ElasticNet
기본 모델	가중치 축소	특성 선택	L1+L2 결합

ElasticNet은 성능이 가장 좋지만 alpha와 l1_ratio 두 개를 조정해야 합니다.

실습 코드 — 최신 scikit-learn API

데이터: fetch_california_housing (인터넷) 또는 load_diabetes (내장)

```
from sklearn.datasets import fetch_california_housing
from sklearn.pipeline import make_pipeline
from sklearn.preprocessing import StandardScaler
from sklearn.neighbors import KNeighborsRegressor
from sklearn.model_selection import train_test_split
X, y = fetch_california_housing(return_X_y=True)
X_tr, X_te, y_tr, y_te = train_test_split(X, y, random_state=42)
knn = make_pipeline(StandardScaler(), KNeighborsRegressor(5)).fit(X_tr, y_tr)
```

주의 사항

load_boston()은 sklearn 1.2에서 제거
→ fetch_california_housing() / load_diabetes()

KNN 실습 3포인트

① 반드시 스케일링 (Pipeline으로 누수 방지)
② random_state=42 고정 · 교차 검증

⚠ sklearn ≥ 1.4 기준 코드 | 전체 예제는 과정 저장소 실습 노트북에 있습니다.

재현성 · 실습 환경 체크리스트

① 데이터

내장 데이터: load_diabetes / fetch_california_housing
과정 저장소: bike/train.csv 로도 실습 가능

② 라이브러리 버전

Python 3.11+
scikit-learn ≥ 1.4 · pandas ≥ 2.0
numpy · matplotlib · mglearn

③ 재현 요령

random_state=42 고정
Pipeline으로 전처리·모델 일괄 적용
교차 검증(cross_val_score)으로 확인

실습 노트북: ch02_01_01A_knn · ch02_01_02A_linear · ch02_01_03_ridge_lasso.ipynb

정형 데이터의 최신 트렌드 (2024–2026)

Gradient Boosting 강세

정형 데이터 상위권은 LightGBM · XGBoost · CatBoost
→ 이 과정의 다음 챕터(앙상블)에서 다룹니다

TabPFN 등장

트랜스포머 사전학습 기반 정형 데이터 모델
소규모 표 데이터에서도 준수한 분류·회귀

KNN의 재발견

임베딩 공간의 최근접 이웃 검색으로 해석
벡터 DB · 추천 · RAG 등에서 활용

정규화 회귀는 여전히 기준선

해석 가능한 모델이 필요하면 릿지 · 라쏘 · ElasticNet
알고리즘보다 데이터 정리와 검증이 더 중요

기본기는 동일 — 정렬 · 스케일링 · 정규화 · 검증을 먼저 익히세요.

SUMMARY

과대적합 보완 기법 총정리

정규화 (L1/L2)

가중치에 패널티를 부여해 복잡도 제어

데이터 증강

이미지·텍스트 등 훈련 데이터 다양하게 확장

앙상블 (Ensemble)

여러 모델을 결합해 안정적인 예측

교차 검증 (CV)

데이터 분할 반복으로 일반화 성능 확인

Early Stopping: 검증 세트 성능이 정체·하락하기 시작하면 학습을 중단합니다.

GUIDE

어떤 알고리즘을 써야 할까?

데이터 적고
빠르게 시작?

→ KNN

직관적, 입문 추천

선형 관계 있고
해석이 필요?

→ 선형 회귀

매개변수 없음

과대적합 걱정?
특성 많음?

→ 릿지 (L2)

기본 선택지

불필요한
특성 제거?

→ 라쏘 (L1)

특성 선택 효과

최고 성능을 원한다면 → **ElasticNet (L1+L2 결합)** — 단, alpha와 l1_ratio 두 개 조정 필요

오늘 배운 것 & 다음 시간

오늘 배운 것

머신러닝 — 지도학습 / 비지도학습
분류 vs 회귀 개념 이해
KNN 알고리즘 원리 & 장단점
선형 회귀 — MSE, R^2
릿지 (L2) · 라쏘 (L1) · ElasticNet

다음 시간 예고

결정 트리 (Decision Tree)
랜덤 포레스트 (Random Forest)
앙상블 학습 기법 심화

오늘 배운 알고리즘들을 scikit-learn으로 직접 실습해 보세요!